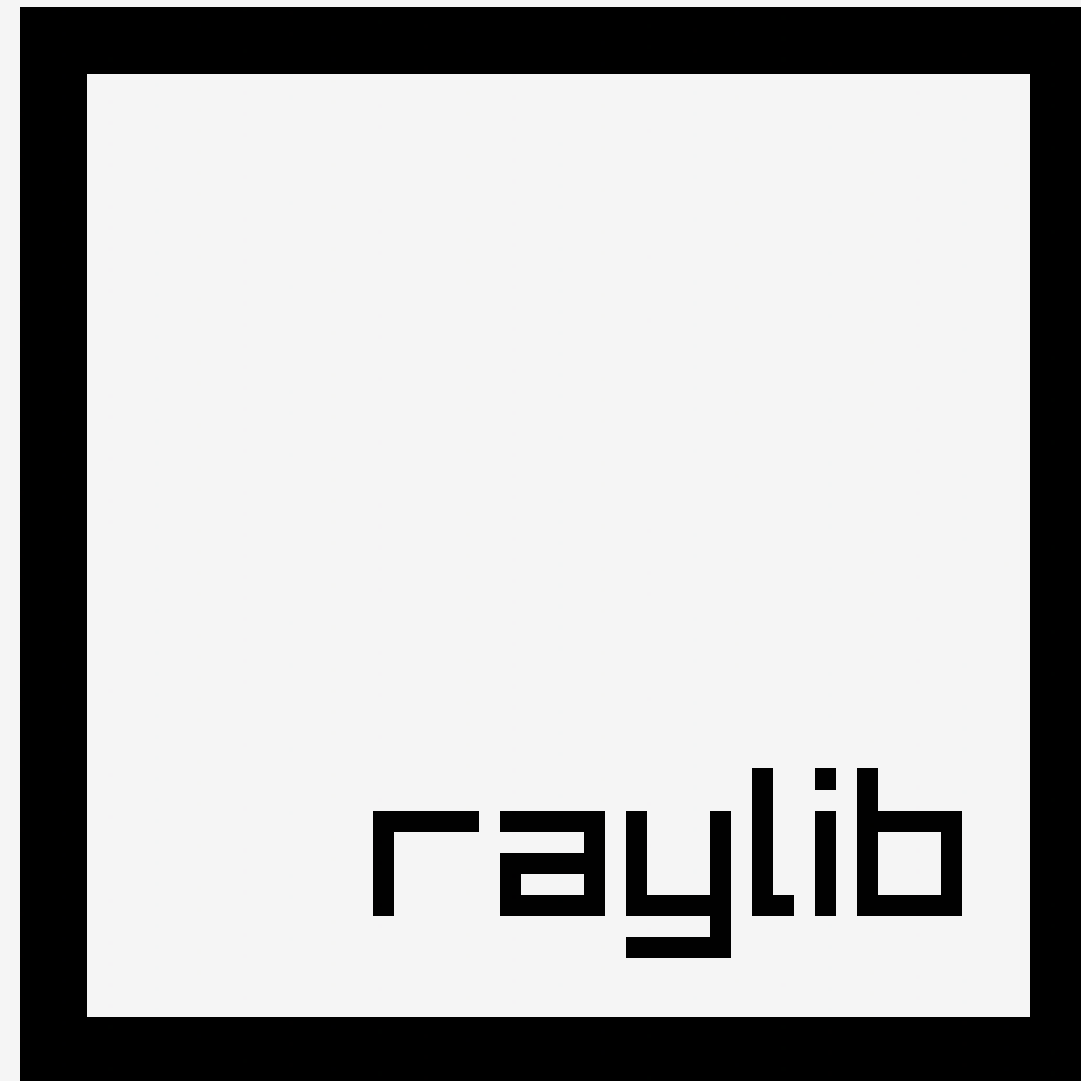


PROGRAMAÇÃO DE JOGOS EM C



Instrutores: Artur Silveira && Filipe Schausst

INTRODUÇÃO

O QUE É UMA BIBLIOTECA ?

- Conjunto de códigos pré-escritos que são utilizados para desenvolver tarefas específicas e repetitivas.
- Economiza tempo
- Resolve problemas



O QUE É RAYLIB ?



- Uma biblioteca simples de usar para desfrutar de programação de jogos.
- Descrição de funções estão contidas no site, em uma tabela. Devem ser consultadas para saber suas diversas funcionalidades.

```

//
// Input-related functions: keyboard
bool IsKeyPressed(int key);
bool IsKeyPressedRepeat(int key);
bool IsKeyDown(int key);
bool IsKeyReleased(int key);
bool IsKeyUp(int key);
int GetKeyPressed(void);
int GetCharPressed(void);
const char *GetKeyName(int key);
void SetExitKey(int key);

// Input-related functions: gamepads
bool IsGamepadAvailable(int gamepad);
const char *GetGamepadName(int gamepad);
bool IsGamepadButtonPressed(int gamepad, int button);
bool IsGamepadButtonDown(int gamepad, int button);
bool IsGamepadButtonReleased(int gamepad, int button);
bool IsGamepadButtonUp(int gamepad, int button);
int GetGamepadButtonPressed(void);
int GetGamepadAxisCount(int gamepad);
float GetGamepadAxisMovement(int gamepad, int axis);
int SetGamepadMappings(const char *mappings);
void SetGamepadVibration(int gamepad, float leftMotor, float rightMotor, float duration);

// Input-related functions: mouse
bool IsMouseButtonPressed(int button);
bool IsMouseButtonDown(int button);
bool IsMouseButtonReleased(int button);
bool IsMouseButtonUp(int button);
int GetMouseX(void);
int GetMouseY(void);
Vector2 GetMousePosition(void);
Vector2 GetMouseDelta(void);
void SetMousePosition(int x, int y);
void SetMouseOffset(int offsetX, int offsetY);
void SetMouseScale(float scaleX, float scaleY);
float GetMouseWheelMove(void);
Vector2 GetMouseWheelMoveV(void);
void SetMouseCursor(int cursor);

// Input-related functions: touch
int GetTouchX(void);
int GetTouchY(void);
Vector2 GetTouchPosition(int index);
int GetTouchPointId(int index);
int GetTouchPointCount(void);

// Check if a key has been pressed once
// Check if a key has been pressed again
// Check if a key is being pressed
// Check if a key has been released once
// Check if a key is NOT being pressed
// Get key pressed (keycode), call it multiple times for keys queued, returns 0 when the queue is empty
// Get char pressed (unicode), call it multiple times for chars queued, returns 0 when the queue is empty
// Get name of a QWERTY key on the current keyboard layout (eg returns string 'q' for KEY_A on an AZERTY keyboard)
// Set a custom key to exit program (default is ESC)

// Check if a gamepad is available
// Get gamepad internal name id
// Check if a gamepad button has been pressed once
// Check if a gamepad button is being pressed
// Check if a gamepad button has been released once
// Check if a gamepad button is NOT being pressed
// Get the last gamepad button pressed
// Get axis count for a gamepad
// Get movement value for a gamepad axis
// Set internal gamepad mappings (SDL_GameControllerDB)
// Set gamepad vibration for both motors (duration in seconds)

// Check if a mouse button has been pressed once
// Check if a mouse button is being pressed
// Check if a mouse button has been released once
// Check if a mouse button is NOT being pressed
// Get mouse position X
// Get mouse position Y
// Get mouse position XY
// Get mouse delta between frames
// Set mouse position XY
// Set mouse offset
// Set mouse scaling
// Get mouse wheel movement for X or Y, whichever is larger
// Get mouse wheel movement for both X and Y
// Set mouse cursor

// Get touch position X for touch point 0 (relative to screen size)
// Get touch position Y for touch point 0 (relative to screen size)
// Get touch position XY for a touch point index (relative to screen size)
// Get touch point identifier for given index
// Get number of touch points

```

POR QUE SE CHAMA RAYLIB ?



Ramón Santamaria

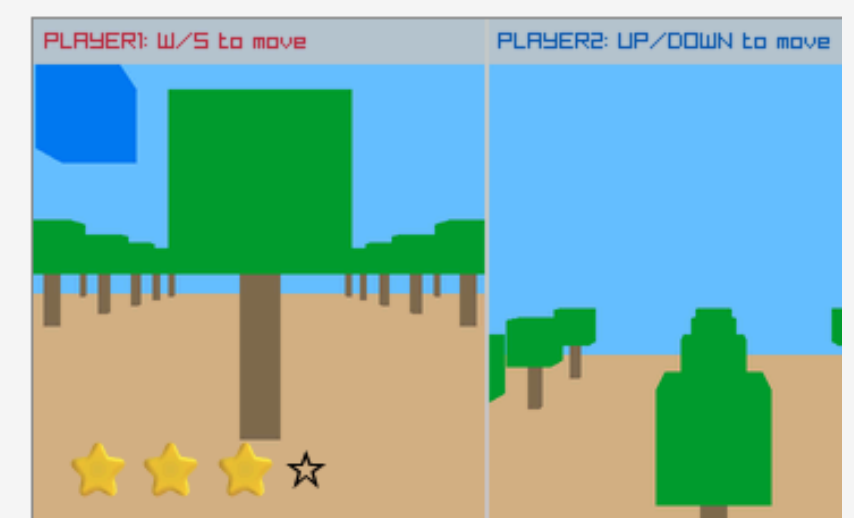
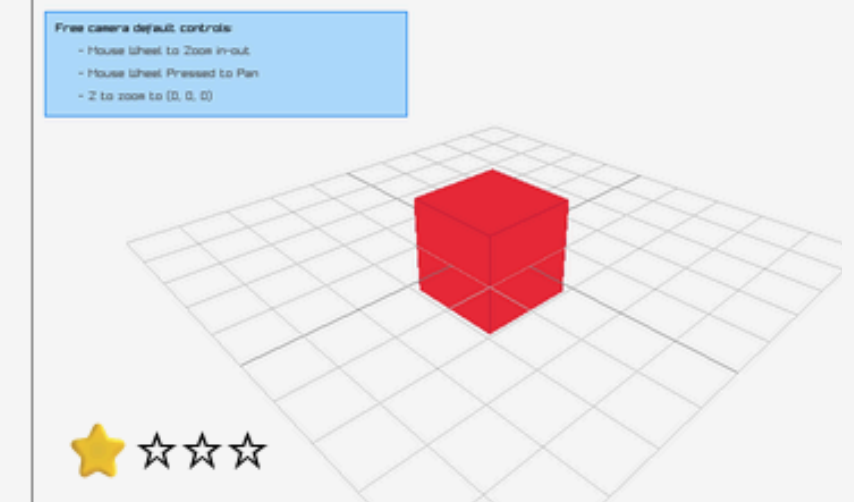
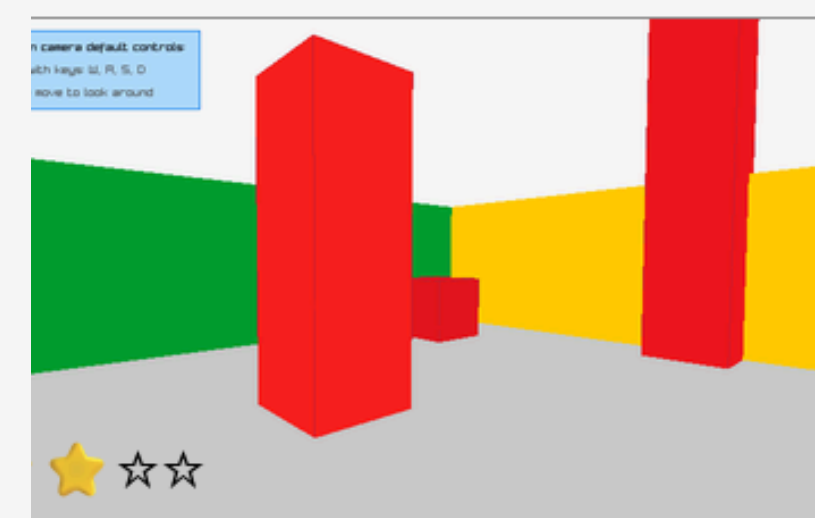
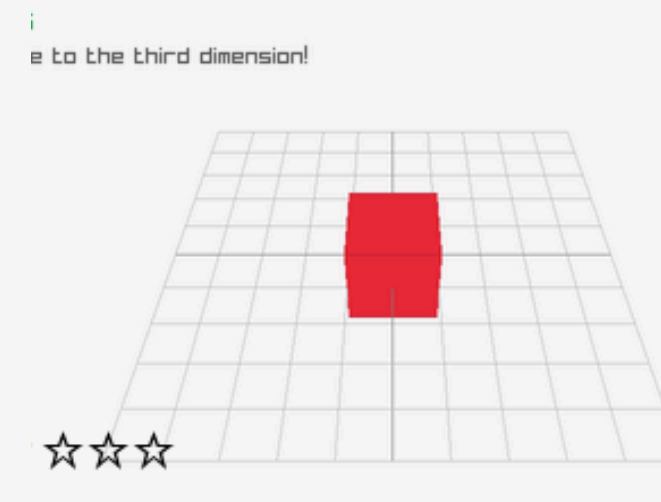
Engenheiro eletrônico, desenvolvedor de software e professor

- Biblioteca criada para permitir que alunos sem o conhecimento de programação possam desenvolver jogos.
- Barcelona (2013).
- Lenda do fliperama.



POSSIBILIDADES DE USO

- Programar jogos 2D
- Programar jogos 3D
- Aplicações GUI
- Editores de audio
- Simulações físicas
- Visualizador de áudio
- Visualizador de áudio
- Painéis interativos
- Aplicações Web



Obs: Raylib pode ser utilizado em Python, C#, C, C++, Go, Rust, Lua e JavaScript .

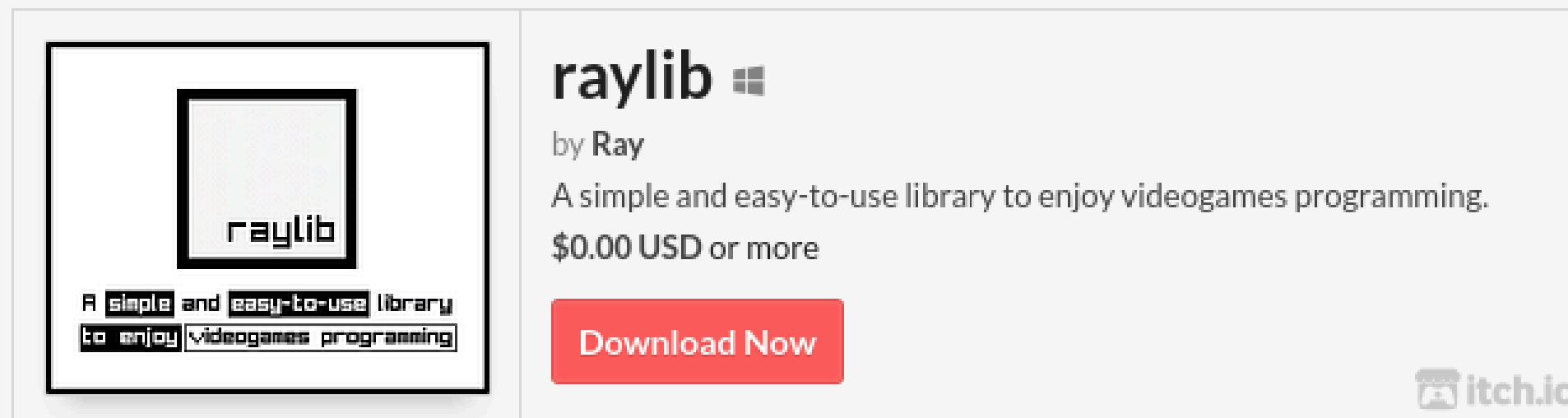
CONCEITOS BÁSICOS

INSTALAÇÃO

Instalador oficial (método mais simples):

Acesse: raylib.com

get the raylib Windows Installer to install raylib in seconds:



NOTE for ADVENTURERS: raylib is a programming library to enjoy videogames programming; no fancy interface, no visual helpers, no gui tools or editors... just coding in pure spartan-programmers way. Are you ready to enjoy coding?

INSTALAÇÃO

Usando Code::Blocks



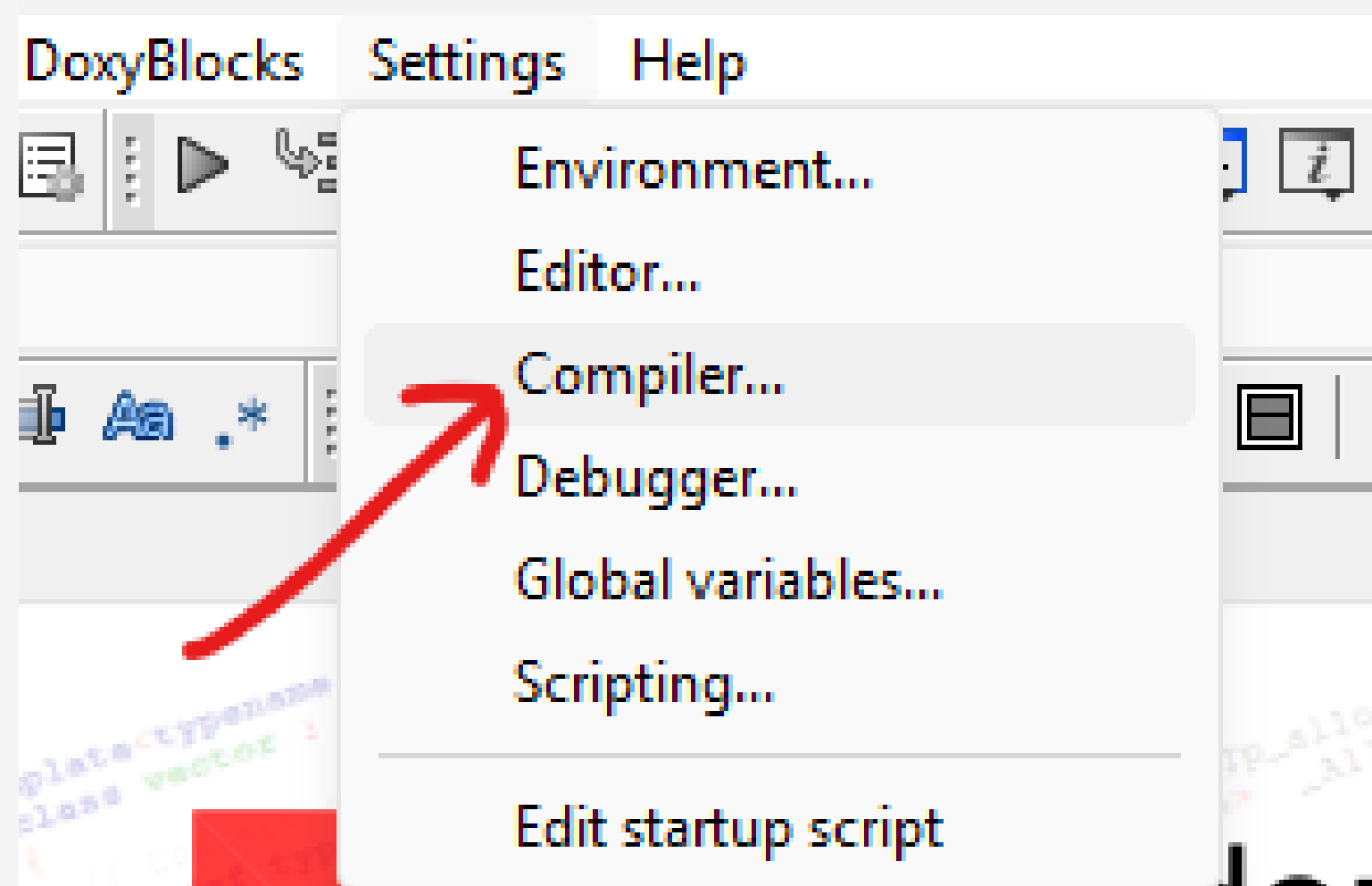
**[github.com/filipe-
dengue/raylib-
codeblocks/](https://github.com/filipedengue/raylib-codeblocks/)**

Link com o kit para
produção dos jogos
usando o Code::Blocks

INSTALAÇÃO

Usando Code::Blocks

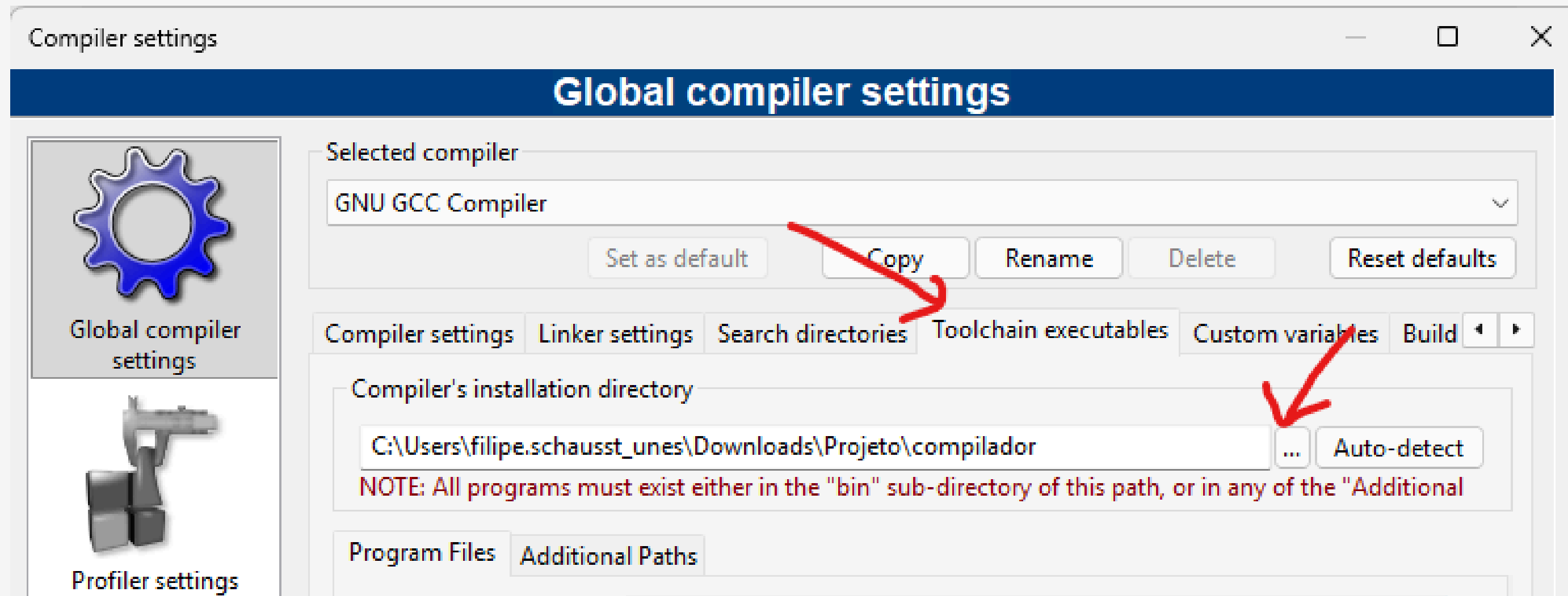
- É necessário mudar o compilador no Code::Blocks



INSTALAÇÃO

Usando Code::Blocks

- É necessário mudar o compilador no Code::Blocks



INSTALAÇÃO

COMPILANDO NO BRAÇO

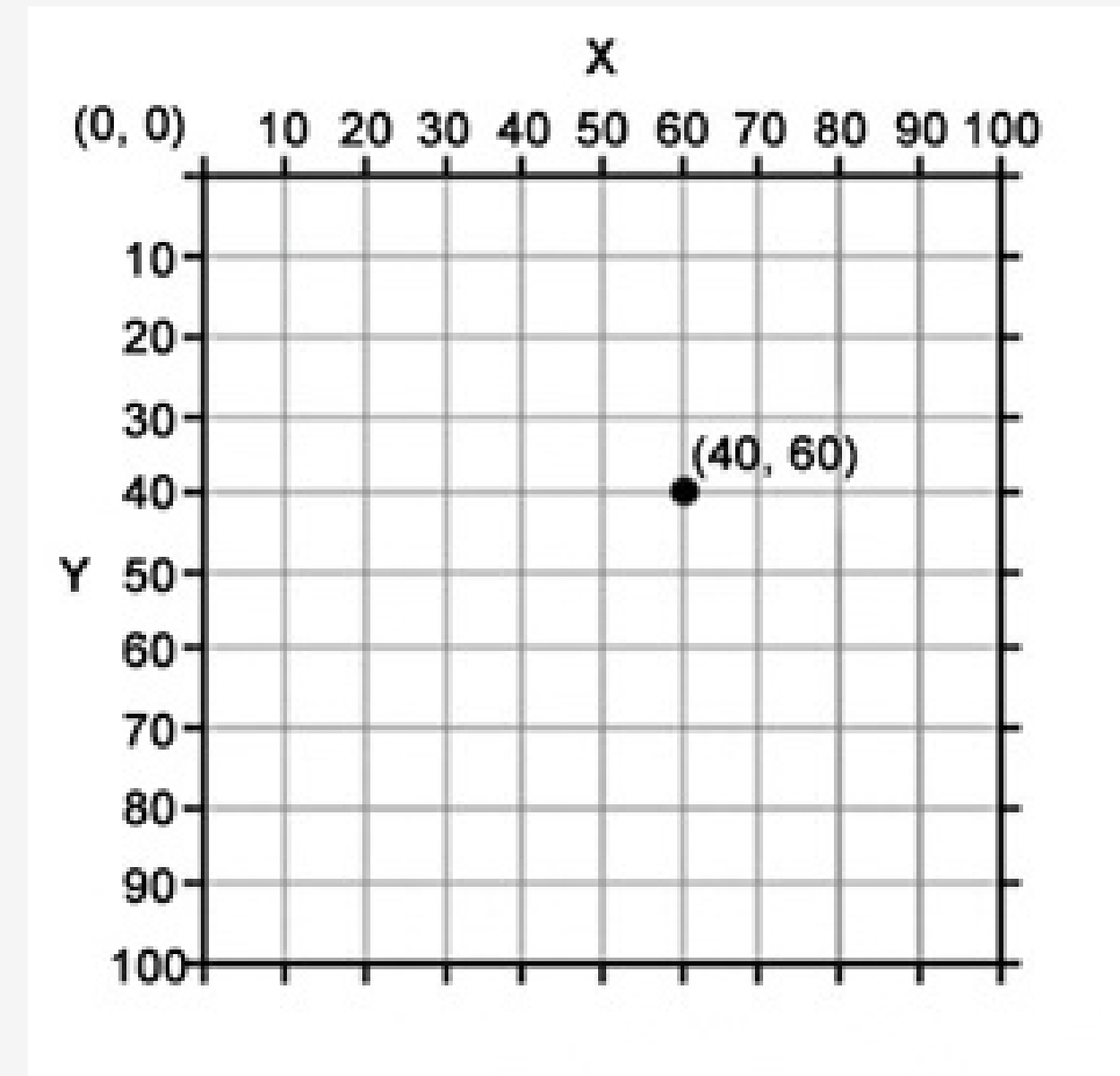
- **gcc**: compilador
- **-o**: renomea o executável
- **jogo.exe**: nome do executável
- **main.c**: código fonte
- **-Iraylib/include**: indica a pasta dos .h
- **-Lraylib/lib**: indica a pasta dos .a (é o código da biblioteca compilado)
- **-lraylib -lgdi32 -lwinmm**: flags do compilador

```
gcc -o jogo.exe main.c -Iraylib/include -Lraylib/lib  
-lraylib -lgdi32 -lwinmm
```

FUNCIONAMENTO BÁSICO

Tela / Sistema de Coordenadas 2D

- Uma tela funciona como um plano cartesiano, onde a posição de um pixel é dado uma coordenada **(x, y)**.
- Lembrando que o **y** aumenta pra baixo.



FUNCIONAMENTO BÁSICO

Loop Do Jogo

- Todo jogo roda em um loop infinito
- Geralmente ele é dividido em duas partes:
 - Atualização da lógica do jogo
 - Atualização dos gráficos

```
InitWindow(800, 600, "Meu Jogo");  
while (!WindowShouldClose()) {  
    // 1. Update (Lógica e Entrada)  
    // 2. Draw (Desenho)  
}  
CloseWindow();
```

C

FUNCIONAMENTO BÁSICO

Structs

- A biblioteca define algumas Structs padrão para facilitar nossa vida

```
typedef struct{  
    float x, y;  
}Vector2;  
  
typedef struct{  
    float x, y, width, height;  
}Rectangle;  
  
typedef struct{  
    unsigned char r, g, b, a;  
}Color;
```

C

FUNCIONAMENTO BÁSICO

Figuras Básicas

- Há várias funções disponíveis para desenhar diversas figuras geométricas
- Geralmente há algumas variações da mesma função.

```
void DrawCircle(int centerX, int centerY, float radius, Color color);  
void DrawCircleV(Vector2 center, float radius, Color color) ;  
  
void DrawRectangle(int posX, int posY, int width, int height, Color color);  
void DrawRectangleV (Vector2 position, Vector2 size, Color color);  
void DrawRectangleRec (Rectangle rec, Color color);  
  
void DrawText (const char *text, int posX, int posY, int fontSize, Color color);
```

C

FUNCIONAMENTO BÁSICO

Entrada de usuário

- Também é bem simples para detectar por alguma entrada de usuário

```
bool IsKeyPressed(int key);
```

- Ideal para detectar quando uma tecla se mantém pressionada

```
bool IsKeyDown (int key);
```

- Retorna quando for teclada

KEY_SPACE

KEY_ENTER

KEY_ESCAPE

KEY_DOWN

KEY_LEFT

KEY_RIGHT

KEY_ZERO

KEY_A

FUNCIONAMENTO BÁSICO

Delta Time

- Útil para realizar movimentação de maneira padrão em todo lugar

```
posicao.x = GetFrameTime() * velocidade;
```

```
posicao.y = GetFrameTime() * velocidade;
```

FUNCIONAMENTO BÁSICO

Detecção de colisão

- Checam se dois elementos diferentes estão dividindo o mesmo espaço

```
bool CheckCollisionCircles(Vector2 center1, float r1, Vector2 center2, float r2);  
  
bool CheckCollisionCircleRec(Vector2 center, float radius, Rectangle rec);  
  
bool CheckCollisionPointRec(Vector2 point, Rectangle rec);  
  
bool CheckCollisionPointCircle(Vector2 point, Vector2 center, float radius) ;  
  
bool CheckCollisionPointTriangle(Vector2 point, Vector2 p1, Vector2 p2, Vector2 p3);
```

PRATICA

FAÇAM O QUE VOCÊS
QUISEREM!

MUITO OBRIGADO!